8

The Stack and Introduction to Procedures

Overview

The stack segment of a program is used for temporary storage of data and addresses. In this chapter we show how the stack can be manipulated, and how it is used to implement procedures.

In section 8.1, we introduce the PUSH and POP instructions that add and remove words from the stack. Because the last word to be added to the stack is the first to be removed, a stack can be used to reverse a list of data; this property is exploited in section 8.2.

Procedures are extremely important in high-level language programming, and the same is true in assembly language. Sections 8.3 and 8.4 discuss the essentials of assembly language procedures. At the machine level, we can see exactly how a procedure is called and how it returns to the calling program. In section 8.5, we present an example of a procedure that performs binary multiplication by bit shifting and addition. This example also gives us an excuse to learn a little more about the DEBUG program.

8.1 The Stack

A *stack* is one-dimensional data structure. Items are added and removed from one end of the structure; that is, it is processed in a "last-in, first-out" manner. The most recent addition to the stack is called the **top of the stack**. A familiar example is a stack of dishes; the last dish to go on the stack is the top one, and it's the only one that can be removed easily.

A program must set aside a block of memory to hold the stack. We have been doing this by declaring a stack segment; for example,

```
.STACK   100H
```

When the program is assembled and loaded in memory, SS will contain the segment number of the stack segment. For the preceding stack declaration, SP, the stack pointer, is initialized to 100h. This represents the empty stack position. When the stack is not empty, SP contains the offset address of the top of the stack.

PUSH and PUSHF

To add a new word to the stack we **PUSH** it on. The syntax is

```
PUSH   source
```

where source is a 16-bit register or memory word. For example,

```
PUSH   AX
```

Execution of PUSH causes the following to happen:

1. SP is decreased by 2.
2. A copy of the source content is moved to the address specified by SS:SP. The source is unchanged.

The instruction **PUSHF**, which has no operands, pushes the contents of the FLAGS register onto the stack.

Initially, SP contains the offset address of the memory location immediately following the stack segment; the first PUSH decreases SP by 2, making it point to the last word in the stack segment. Because each PUSH

Figure 8.1A Empty Stack

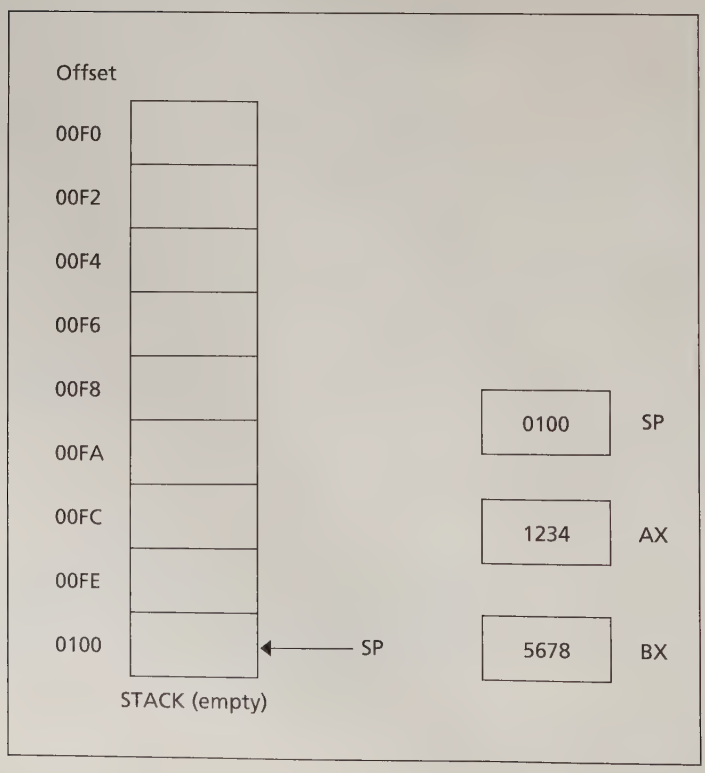
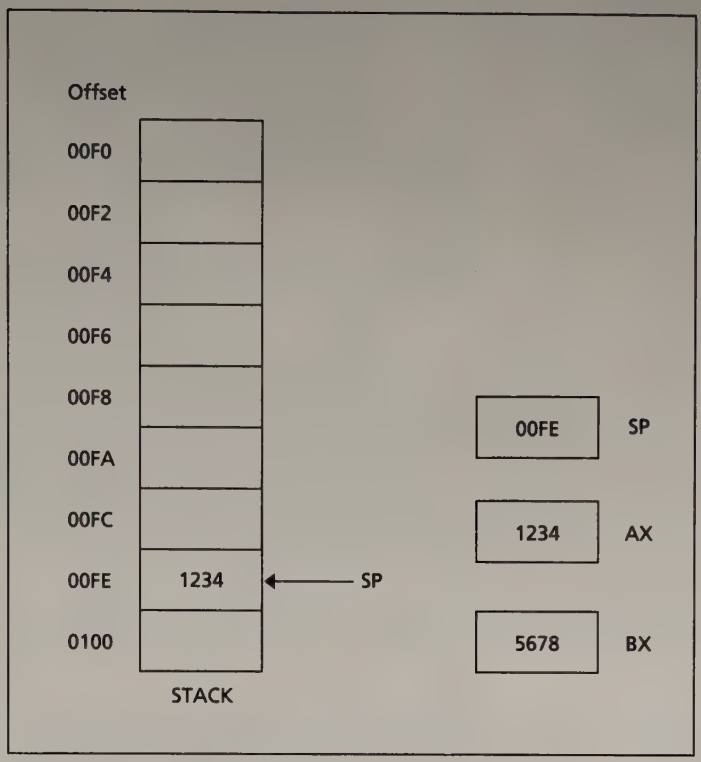
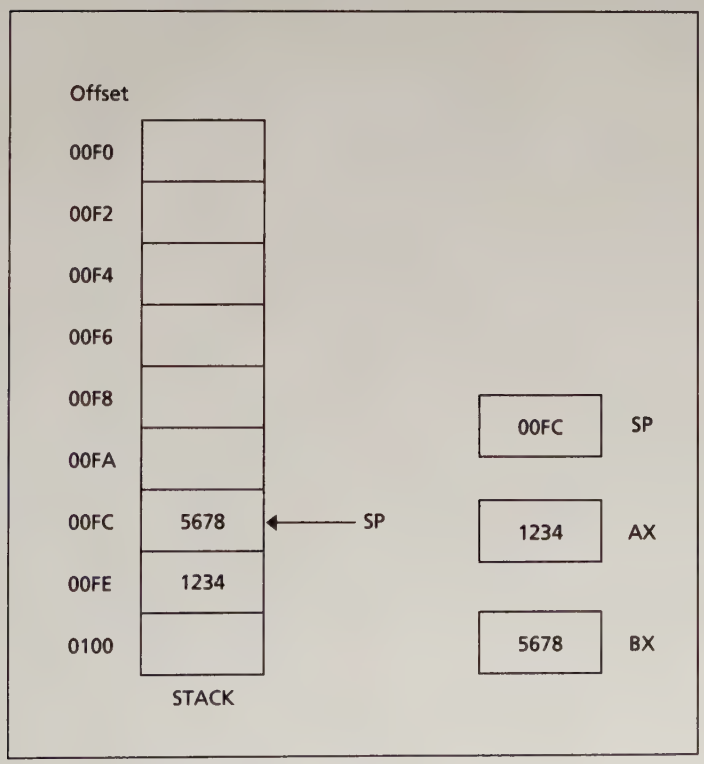


Figure 8.1B After PUSH AX



decreases SP, the stack grows toward the beginning of memory. Figure 8.1 shows how PUSH works.

8.1C After PUSH BX



POP and POPF

To remove the top item from the stack, we **POP** it. The syntax is

```
POP   destination
```

where destination is a 16-bit register (except IP) or memory word. For example,

```
POP   BX
```

Executing POP causes this to happen:

1. The content of SS:SP (the top of the stack) is moved to the destination.
2. SP is increased by 2.

Figure 8.2 shows how POP works.

The instruction **POPF** pops the top of the stack into the FLAGS register. There is no effect of PUSH, PUSHF, POP, POPF on the flags.

Note that PUSH and POP are word operations, so a byte instruction such as

```
Illegal:   PUSH DL
```

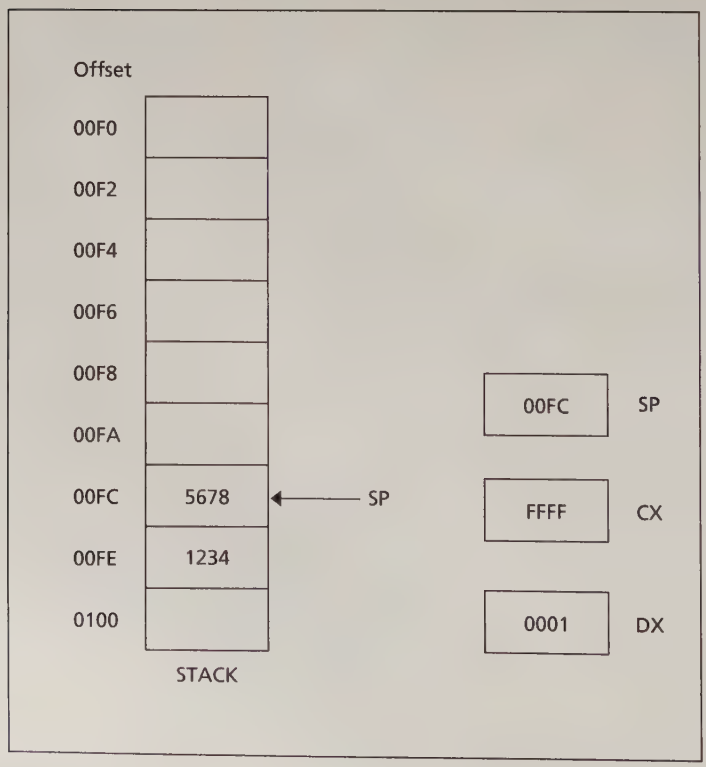
is illegal. So is a push of immediate data, such as

```
Illegal:   PUSH 2
```

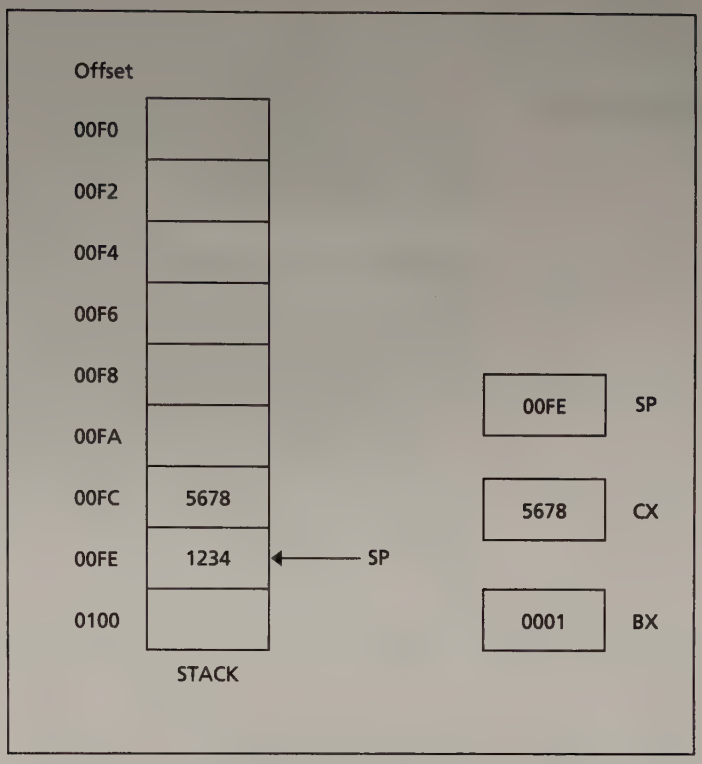
Note: an immediate data push is legal for the 80186/80486 processors. These processors are discussed in Chapter 20.

In addition to the user's program, the operating system uses the stack for its own purposes. For example, to implement the INT 21h functions, DOS saves any registers it uses on the stack and restores them when the interrupt routine is completed. This does not cause a problem for the user

8.2A Before POP

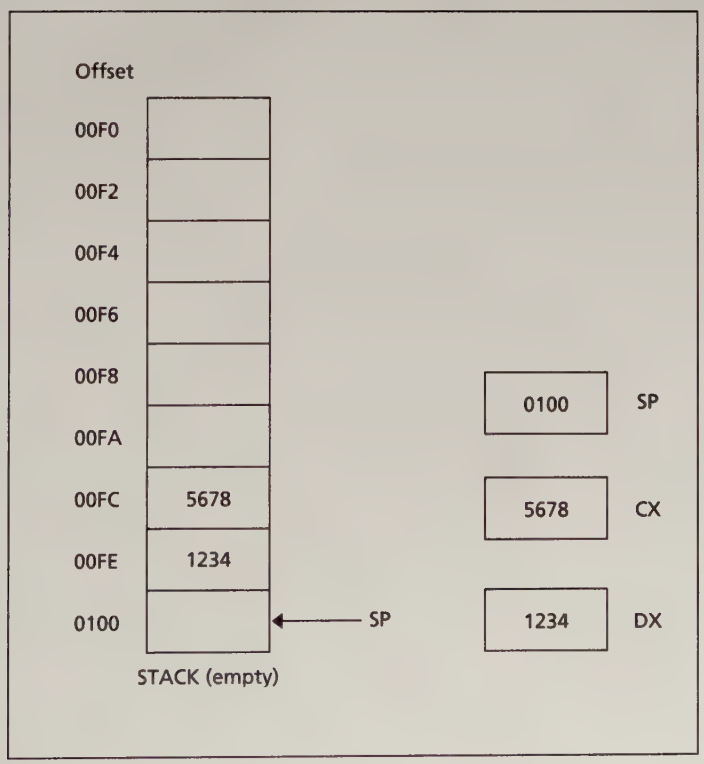


8.2B After POP CX



because any values DOS pushes onto the stack are popped off by DOS before it returns control to the user's program.

8.2C After POP DX



8.2 A Stack Application

Because the stack behaves in a last-in, first-out manner, the order that items come off the stack is the reverse of the order they enter it. The following program uses this property to read a sequence of characters and display them in reverse order on the next line.

Algorithm to Reverse Input

```
Display a '?'
Initialize count to 0
Read a character
WHILE character is not a carriage return DO
   push character onto the stack
   increment count
   read a character
END_WHILE;
Go to a new line
FOR count times DO
   pop a character from the stack;
   display it;
END_FOR
```

Here is the program:

Program Listing PGM8_1.ASM

```
1:  TITLE PGM8_1:REVERSE INPUT
2:  .MODEL     SMALL
3:  .STACK     100H
4:  .CODE
5:  MAIN       PROC
6:  ;display user prompt
7:          MOV    AH,2          ;prepare to display
8:          MOV    DL,'?'        ;char to display
9:          INT    21H           ;display '?'
10: ;initialize character count
11:         XOR    CX,CX         ;count = 0
12: ;read a character
13:         MOV    AH,1          ;prepare to read
14:         INT    21H           ;read a char
15: ;while character is not a carriage return do
16: WHILE_:
17:         CMP    AL,0DH        ;CR?
18:         JE     END_WHILE     ;yes, exit loop
19:         ;save character on the stack and increment count
20:         PUSH   AX            ;push it on stack
21:         INC    CX            ;count = count + 1
22: ;read a character
23:         INT    21H           ;read a char
24:         JMP    WHILE_        ;loop back
25: END_WHILE:
26: ;go to a new line
27:         MOV    AH,2          ;display char fcn
28:         MOV    DL,0DH        ;CR
29:         INT    21H           ;execute
30:         MOV    DL,0AH        ;LF
```

```
31:          INT    21H          ;execute
32:          JCXZ   EXIT         ;exit if no characters read
33: ;for count times do
34: TOP:
35: ;pop a character from the stack
36:          POP    DX           ;get a char from stack
37: ;display it
38:          INT    21H          ;display it
39:          LOOP   TOP
40: ;end_for
41: EXIT:
42:          MOV    AH,4CH
43:          INT    21H
44: MAIN     ENDP
45:          END  MAIN
```

Because the number of characters to be entered is unknown, the program uses CX to count them. CX controls the FOR loop that displays the characters in reverse order.

In lines 16–24, the program executes a WHILE loop that pushes characters on the stack and reads new ones, until a carriage return is entered. Even though the input characters are in AL, it's necessary to save all of AX on the stack, because the operand of PUSH must be a word.

When the program exits the WHILE loop (line 25), all the characters are on the stack, with the low byte of the top of the stack containing the last character to be entered. AL contains the ASCII code of the carriage return.

At line 32, the program checks to see if any characters were read. If not, CX contains 0 and the program jumps to the DOS exit. If any characters *were* read, the program enters a FOR loop that repeatedly pops the stack into DX (so that DL will get a character code), and displays a character.

Sample executions:

```
C>PGM8_1
?THIS IS A TEST
TSET A SI SIHT

C>PGM8_1
?A
A

C>PGM8_1
?      (only carriage return typed)
       (no output)
C>
```

8.3 Terminology of Procedures

In Chapter 6, we mentioned the idea of top-down program design. The idea is to take the original problem and decompose it into a series of subproblems that are easier to solve than the original problem. High-level languages usually employ procedures to solve these subproblems, and we can do the same thing in assembly language. Thus an assembly language program can be structured as a collection of procedures.

One of the procedures is the main procedure, and it contains the entry point to the program. To carry out a task, the main procedure **calls** one of the other procedures. It is also possible for these procedures to call each other, or for a procedure to call itself.

When one procedure calls another, control transfers to the called procedure and its instructions are executed; the called procedure usually returns control to the caller at the next instruction after the call statement (Figure 8.3). For high-level languages, the mechanism by which call and return are implemented is hidden from the programmer, but in assembly language we can see how it works (see section 8.4).

Procedure Declaration

The syntax of procedure declaration is the following:

```
name    PROC   type
;body of the procedure
        RET
name    ENDP
```

Name is the user-defined name of the procedure. The optional operand type is **NEAR** or **FAR** (if type is omitted, NEAR is assumed). NEAR means that the statement that calls the procedure is in the same segment as the procedure itself; FAR means that the calling statement is in a different segment. In the following, we assume all procedures are NEAR; FAR procedures are discussed in Chapter 14.

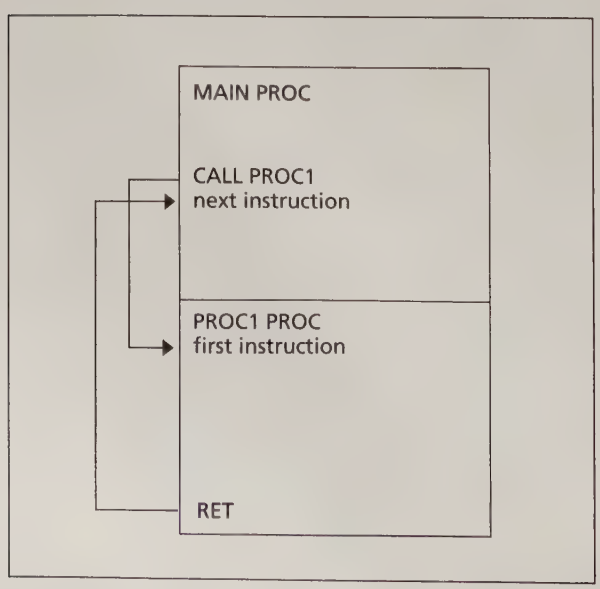


Figure 8.3 Procedure Call and Return

RET

The **RET** (return) instruction causes control to transfer back to the calling procedure. Every procedure (except the main procedure) should have a RET someplace; usually it's the last statement in the procedure.

Communication Between Procedures

A procedure must have a way to receive values from the procedure that calls it, and a way to return results. Unlike high-level language procedures, assembly language procedures do not have parameter lists, so it's up to the programmer to devise a way for procedures to communicate. For example, if there are only a few input and output values, they can be placed in registers. The general issue of procedure communication is discussed in Chapter 14.

Procedure Documentation

In addition to the required procedure syntax, it's a good idea to document a procedure so that anyone reading the program listing will know what the procedure does, where it gets its input, and where it delivers its output. In this book, we generally document procedures with a comment block like this:

```
; (describe what the procedure does)
; input:  (where it receives information from
           the calling program)
; output: (where it delivers results to
           the calling program)
; uses:   (a list of procedures that it calls)
```

8.4 CALL and RET

To invoke a procedure, the **CALL** instruction is used. There are two kinds of procedure calls, **direct** and **indirect**. The syntax of a direct procedure call is

```
CALL   name
```

where name is the name of a procedure. The syntax of an indirect procedure call is

```
CALL   address_expression
```

where address_expression specifies a register or memory location containing the address of a procedure.

Executing a CALL instruction causes the following to happen:

1. The return address to the calling program is saved on the stack. This is the offset of the next instruction after the CALL statement. The segment:offset of this instruction is in CS:IP at the time the call is executed.

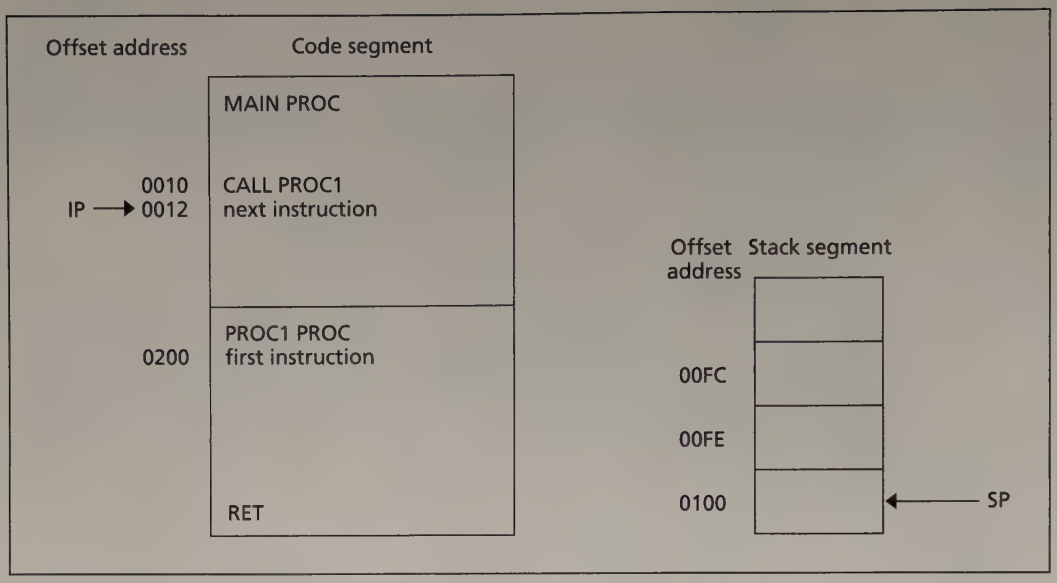


Figure 8.4A Before CALL

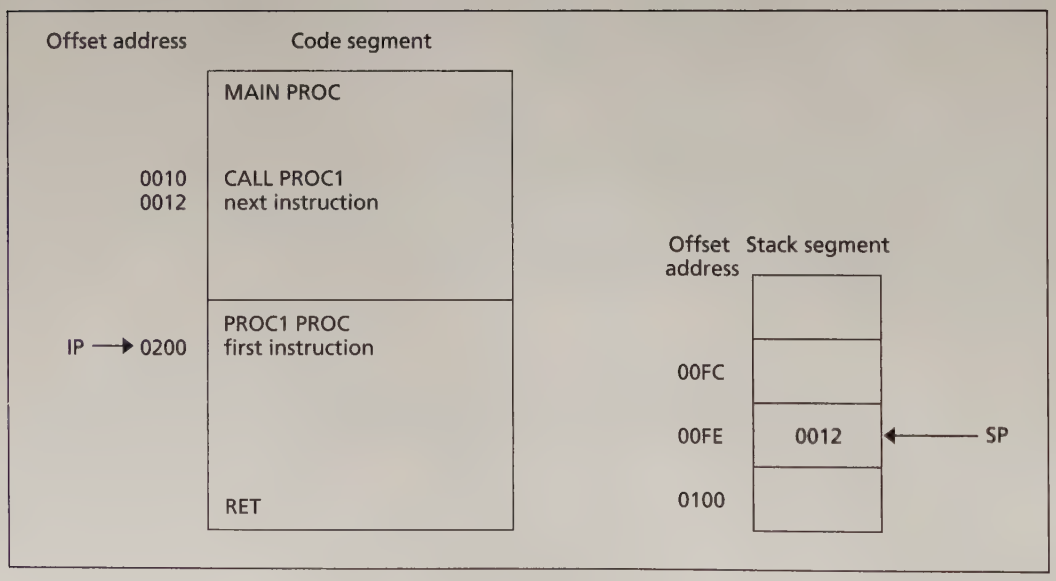


Figure 8.4B After CALL

2. IP gets the offset address of the first instruction of the procedure. This transfers control to the procedure. See Figures 8.4A and 8.4B.

To return from a procedure, the instruction

```
RET   pop_value
```

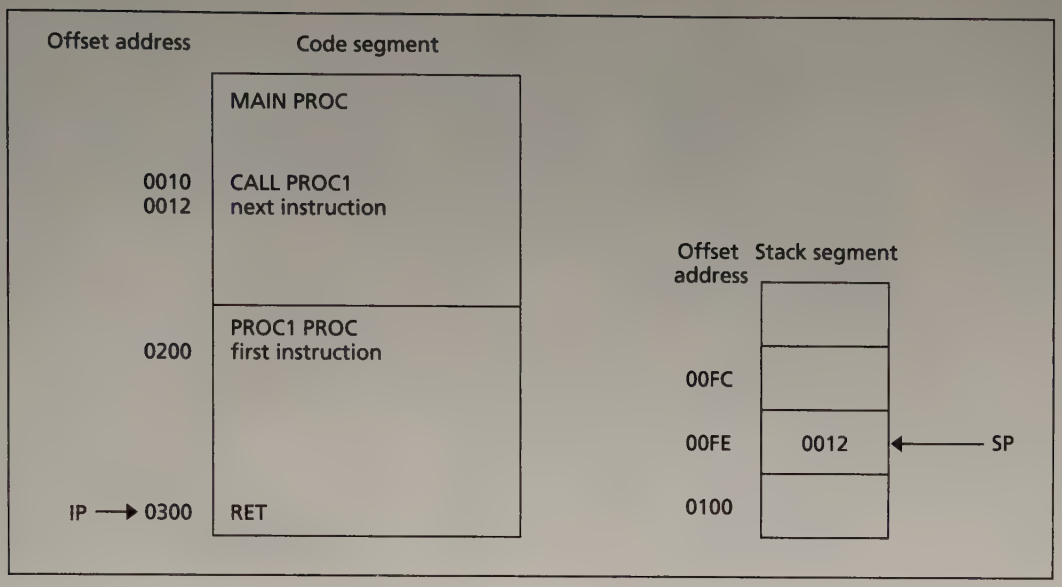


Figure 8.5A Before RET

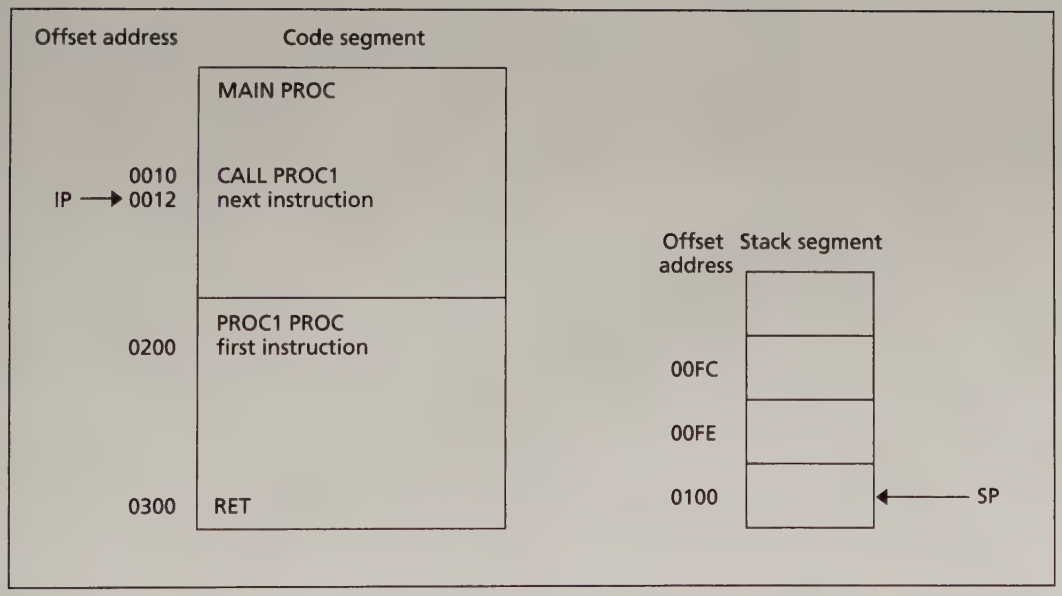


Figure 8.5B After RET

is executed. The integer argument pop_value is optional. For a NEAR procedure, execution of RET causes the stack to be popped into IP. If a pop_value *N* is specified, it is added to SP, and thus has the effect of removing *N* additional bytes from the stack. CS:IP now contains the segment:offset of the return address, and control returns to the calling program. See Figures 8.5A and 8.5B.

8.5 An Example of a Procedure

As an example, we will write a procedure for finding the product of two positive integers A and B by addition and bit shifting. This is one way unsigned multiplication may be implemented on the computer (in Chapter 9 we introduce the multiplication instructions).

Multiplication algorithm:

```
Product = 0
REPEAT
   IF lsb of B is 1   (Recall lsb = least
             significant bit)
   THEN
      Product = Product + A
   END_IF
   Shift left A
   Shift right B
UNTIL B = 0
```

For example, if A = 111b = 7 and B = 1101b = 13

```
Product = 0
Since lsb of B is 1, Product = 0 + 111b = 111b
Shift left A: A = 1110b
Shift right B: B = 110b

Since lsb of B is 0,
Shift left A: A = 11100b
Shift right B: B = 11b

Since lsb of B is 1
Product = 111b + 11100b = 100011b
Shift left A: A = 111000b
Shift right B: B = 1

Since lsb of B is 1
Product = 100011b + 111000b = 1011011b
Shift left A: A = 1110000b
Shift right B: B = 0

Since lsb of B = 0
Return Product = 1011011b = 91d
```

Note that we get the same answer by performing the usual decimal multiplication process on the binary numbers:

```
    111b
  x 1101b
  -------
     111
    000
   111
  111
  -------
  1011011b
```

In the following program, the algorithm is coded as a procedure MULTIPLY. The main program has no input or output; we will use DEBUG for the I/O.

Program Listing PGM8_2.ASM

```
1:  TITLE    PGM8_2: MULTIPLICATION BY ADD AND SHIFT
2:  .MODEL    SMALL
3:  .STACK    100H
4:  .CODE
5:  MAIN     PROC
6:  ;execute in DEBUG. Place A in AX and B in BX
7:           CALL   MULTIPLY
8:  ;DX will contain the product
9:           MOV    AH,4CH
10:          INT    21H
11: MAIN     ENDP
12: MULTIPLY PROC
13: ;multiplies two nos. A and B by shifting and addition
14: ;input:   AX = A, BX = B. Nos. in range 0 - FFh
15: ;output:  DX = product
16:          PUSH   AX
17:          PUSH   BX
18:          XOR    DX,DX         ;product = 0
19: REPEAT:
20: ;if B is odd
21:          TEST   BX,1          ;is B odd?
22:          JZ     END_IF        ;no, even
23: ;then
24:          ADD    DX,AX         ;prod = prod + A
25: END_IF:
26:          SHL    AX,1          ;shift left A
27:          SHR    BX,1          ;Shift right B
28: ;until
29:          JNZ    REPEAT
30:          POP    BX
31:          POP    AX
32:          RET
33: MULTIPLY ENDP
34:          END   MAIN
```

Procedure MULTIPLY receives its input A and B through registers AX and BX, respectively. Values are placed in these registers by the user inside the DEBUG program; the product is returned in DX. In order to avoid overflow, A and B are restricted to range from 0 to FFh.

A procedure usually begins by saving all the registers it uses on the stack and ends by restoring these registers. This is done because the calling program may have data stored in registers, and the actions of the procedure could cause unwanted side effects if the registers are not preserved. Even though it's not really necessary in this program, we illustrate this practice by pushing AX and BX on the stack in lines 16 and 17, and restoring them in lines 30 and 31. The registers are popped off the stack in the reverse order that they were pushed on.

After clearing DX, which will hold the product, the procedure enters a REPEAT loop (lines 19–29). At line 22, the procedure checks BX's least significant bit. If the lsb of BX is 1, then AX is added to the product in DX; if the lsb of BX is 0, the procedure skips to line 26. Here AX is shifted left, and BX is shifted right; the loop continues until BX = 0. The procedure exits with the product in DX.

After assembling and linking the program, we take it into DEBUG (in the following, the user's response appears in boldface):

```
C> DEBUG PGM8_2.EXE
-
```

DEBUG responds with its command prompt "-". To get a listing of the program, we use the U (unassemble) command.

```
-U
177F:0000 E80400        CALL    0007
177F:0003 B44C          MOV     AH,4C
177F:0005 CD21          INT     21
177F:0007 50            PUSH    AX
177F:0008 53            PUSH    BX
177F:0009 33D2          XOR     DX,DX
177F:000B F7C30100      TEST    BX,0001
177F:000F 7402          JZ      0013
177F:0011 03D0          ADD     DX,AX
177F:0013 D1E0          SHL     AX,1
177F:0015 D1EB          SHR     BX,1
177F:0017 75F2          JNZ     000B
177F:0019 5B            POP     BX
177F:001A 58            POP     AX
177F:001B C3            RET
177F:001C E3D1          JCXZ    FFEF
177F:001E E38B          JCXZ    FFAB
```

The U command causes DEBUG to interpret the contents of memory as machine language instructions. The display gives the segment:offset of each instruction, the machine code, and the assembly code. All numbers are expressed in hex. From the first statement, CALL 0007, we can see that procedure MAIN extends from 0000 to 0005; procedure MULTIPLY begins at 0007 and ends at 001B with RET. The instructions after this are garbage.

Before entering the data, let's look at the registers.

```
-R
AX=0000  BX=0000  CX=001C  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0000  NV UP EI PL NZ NA PO NC
177F:0000 E80400   CALL  0007
```

The initial value of SP = 100h reflects that fact that we allocated 100h bytes for the stack. To have a look at the empty stack, we can dump memory with the D command.

```
DSS:F0 FF
1781:00F0  00 00 00 00 00 00 6F 17-A4 13 07 00 6F 17 00 00
```

The command DSS:F0 FF means to display the memory bytes from SS:F0 to SS:FF. This is the last 16 bytes in the stack segment. The contents of each byte is displayed as two hex digits. Because the stack is empty, everything in this display is garbage.

Before executing the program, we need to place the numbers A and B in AX and BX, respectively. We will use A = 7 and B = 13 = Dh. To enter A, we use the R command:

```
-RAX
AX 0000:7
```

The command RAX means that we want to change the content of AX. DEBUG displays the current value (0000), followed by a colon, and waits for us to enter the new value. Similarly we can change the initial value of B in BX:

```
-RBX
BX 0000:D
```

Now let's look at the registers again.

```
-R
AX=0007  BX=000D  CX=001C  DX=0000  SP=0100  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0000  NV UP EI PL NZ NA PO NC
177F:0000 E80400   CALL  0007
```

We see that AX and BX now contain the initial values.

To see the effect of the first instruction, CALL 0007, we use the T (trace) command. It will execute a single instruction and display the registers.

```
-T
AX=0007  BX=000D  CX=001C  DX=0000  SP=00FE  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0007  NV UP EI PL NZ NA PO NC
177F:0007 50    PUSH  AX
```

We notice two changes in the registers: (1) IP now contains 0007, the starting offset of procedure MULTIPLY; and (2) because the CALL instruction pushes the return address to procedure MAIN on the stack, SP has decreased from 0100h to 00FEh. Here are the last 16 bytes of the stack segment again:

```
-DSS:F0 FF
1781:00F0 00 00 00 00 07 00 00 00-07 00 7F 17 A4 13 03 00
```

The return address is 0003, but is displayed as 03 00. This is because DEBUG displays the low byte of a word before the high byte.

The first three instructions of procedure MULTIPLY push AX and BX onto the stack, and clear DX. To see the effect, we use the **G** (go) command. The syntax is

```
G offset
```

It causes the program to execute instructions and stop at the specified offset. From the unassembled listing given earlier, we can see that the next instruction after XOR DX,DX is at offset 000Bh.

```
-GB
AX=0007  BX=000D  CX=001C  DX=0000  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=000B  NV UP EI PL ZR NA PE NC
177F:000B F7C30100    TEST  BX,0001
```

We see that the two PUSHes have caused SP to decrease by 4, from 00FEh to 00FAh. Now the stack looks like this:

```
-DSS:F0 FF
1781:00F0  00 00 00 00 07 00 00 17-A4 13 0D 00 07 00 03 00
```

The stack now contains three words; the values of BX (000D), AX (0007), and the return address (0003). These are shown as 0D 00 07 00 03 00.

Now let's watch the procedure in action. To do so, we will execute to the end of the REPEAT loop at offset 0017h:

```
-G17
AX=000E  BX=0006  CX=001C  DX=0007  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0017  NV UP EI PL NZ AC PE CY
177F:0017 75F2    JNZ 000B
```

Because the initial value of B in BX was 0Dh = 1101b, the lsb of BX is 1, so AX is added to the product in DX, giving 111b = 0007h. AX is shifted left, which doubles A to 14d = 000Eh, and BX is shifted right, which halves BX (and rounds down) to 0006h = 110b.

To get to the top of the loop, we'll use the T command again:

```
-T
AX=000E  BX=0006  CX=001C  DX=0007  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=000B  NV UP EI PL NZ AC PE CY
177F:000B F7C30100    TEST BX,0001
```

and execute again to the bottom:

```
-G17
AX=001C  BX=0003  CX=001C  DX=0007  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0017  NV UP EI PL NZ AC PE NC
177F:0017 75F2    JNZ 000B
```

Because BX = 0006h = 110b, the lsb of BX is 0, so the product in DX stays the same. AX is shifted left to 11100b = 1Ch and BX is shifted right to 11b = 3h.

After two more trips through the loop, the product is in DX. Watch AX, BX, and DX change:

```
-T
AX=001C  BX=0003  CX=001C  DX=0007  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=000B  NV UP EI PL NZ AC PE NC
177F:000B F7C30100    TEST BX,0001

-G17
AX=0038  BX=0001  CX=001C  DX=0023  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0017  NV UP EI PL NZ AC PO CY
177F:0017 75F2    JNZ 000B
```

```
-T
AX=0038  BX=0001  CX=001C  DX=0023  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=000B  NV UP EI PL NZ AC PO CY
177F:000B F7C30100    TEST BX,0001

-G17
AX=0070  BX=0000  CX=001C  DX=005B  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0017  NV UP EI PL ZR AC PE CY
177F:0017 75F2     JNZ 000B
```

The last right shift made BX = 0, ZF = 1, so the loop ends. The product = 91 = 5Bh is in DX.

To terminate the procedure, we trace through the JNZ and the two POP instructions:

```
-T
AX=0070  BX=0000  CX=001C  DX=005B  SP=00FA  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0019  NV UP EI PL ZR AC PE CY
177F:0019 5B    POP BX

-T
AX=0070  BX=000D  CX=001C  DX=005B  SP=00FC  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=001A  NV UP EI PL ZR AC PE CY
177F:001A 58    POP AX

-T
AX=0007  BX=000D  CX=001C  DX=005B  SP=00FE  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=001B  NV UP EI PL ZR AC PE CY
177F:001B C3    RET
```

The two POPs have restored AX and BX to their original values. Let's look at the stack:

```
-DSS:F0 FF
1781:00F0  70 00 70 00 07 00 00 00-1B 00 7F 17 A4 13 03 00
```

The values 000D and 0007 are no longer in the display. This is not a result of the POP instruction; it's because DEBUG is also using the stack.

Finally, we trace the RET.

```
-T
AX=0007  BX=000D  CX=001C  DX=005B  SP=0100  BP=0000  SI=0000  DI=0000
DS=176F  ES=176F  SS=1781  CS=177F  IP=0003  NV UP EI PL ZR AC PE CY
177F:0003 B44C    MOV AH,4C
```

RET causes IP to get 0003, the return address to MAIN. SP goes back to 100h, its original value. To finish executing the program, we just type G:

```
-G
Program terminated normally
```

and we exit DEBUG by typing Q (quit).

```
-Q
>C
```

Summary

- The stack is a temporary storage area used by both application programs and the operating system.
- The stack is a last-in, first-out data structure. SS:SP points to the top of the stack.
- The stack-altering instructions are PUSH, PUSHF, POP, and POPF. PUSH adds a new top word to the stack, and POP removes the top word. PUSHF saves the FLAGS register on the stack and POPF puts the stack top into the FLAGS register.
- SP decreases by 2 when PUSH or PUSHF is executed, and it increases by 2 when POP or POPF is executed. SP is initialized to the first word after stack segment when the program is loaded.
- A procedure is a subprogram. Assembly language programs are typically broken into two procedures. One of the procedures is the main procedure, which contains the entry point to the program. Procedures may call other procedures, or themselves.
- There are two kinds of procedures, NEAR and FAR. A NEAR procedure is in the same code segment as the calling program, and a FAR procedure is in a different segment.
- The CALL instruction is used to invoke a procedure. For a NEAR procedure, execution of CALL causes the offset address of the next instruction in line after the CALL to be saved on the stack, and the IP gets the offset of the first instruction in the procedure.

- Procedures end with a RET instruction. Its execution causes the stack to be popped into IP, and control returns to the calling program. In order for the return address to be accessible, the procedure must ensure that it is at the top of the stack when RET is executed.
- In assembly language, procedures often pass data through registers.

Glossary

direct procedure call	A procedure call of form CALL name
FAR procedure	A procedure that can be called by procedures residing in any segment
indirect procedure call	A procedure call of form CALL addr_exp
NEAR procedure	A procedure that can only be called by another procedure residing in the same segment
top of the stack	The last word of data added to the stack

New Instructions

CALL	POPF	PUSHF
POP	PUSH	RET

Exercises

1. Suppose the stack segment is declared as follows:

```
.STACK  100h
```

   a. What is the hex contents of SP when the program begins?
   b. What is the maximum hex number of words that the stack may contain?

2. Suppose that AX = 1234h, BX = 5678h, CX = 9ABCh, and SP = 100h. Give the contents of AX, BX, CX, and SP after executing the following instructions:

```
PUSH  AX
PUSH  BX
XCHG  AX,CX
POP   CX
PUSH  AX
POP   BX
```

3. When the stack has completely filled the stack area, SP = 0. If another word is pushed onto the stack, what would happen to SP? What might happen to the program?
4. Suppose a program contains the lines

```
CALL  PROC1
MOV   AX,BX
```

   and (a) instruction MOV AX,BX is stored at 08FD:0203h, (b) PROC1 is a NEAR procedure that begins at 08FD:300h, (c) SP = 010Ah. What are the contents of IP and SP just after CALL PROC1 is executed? What word is on top of the stack?

5. Suppose SP = 0200h, top of stack = 012Ah. What are the contents of IP and SP
   a. after RET is executed, where RET appears in a NEAR procedure?
   b. after RET 4 is executed, where RET appears in a NEAR procedure?
6. Write some code to
   a. place the top of the stack into AX, without changing the stack contents.
   b. place the word that is below the stack top into CX, without changing the stack contents. You may use AX.
   c. exchange the top two words on the stack. You may use AX and BX.
7. Procedures are supposed to return the stack to the calling program in the same condition that they received it. However, it may be useful to have procedures that alter the stack. For example, suppose we would like to write a NEAR procedure SAVE_REGS that saves BX,CX,DX,SI,DI,BP,DS, and ES on the stack. After pushing these registers, the stack would look like this:

```
ES content
.
.
.
DX content
CX content
BX content
return_address (offset)
```

   Now, unfortunately, SAVE_REGS can't return to the calling program, because the return address is not at the top of the stack.
   a. Devise a way to implement a procedure SAVE_REGS that gets around this problem (you may use AX to do this).
   b. Write a procedure RESTORE_REGS that restores the registers that SAVE_REGS has saved.

Programming Exercises

8. Write a program that lets the user type some text, consisting of words separated by blanks, ending with a carriage return, and displays the text in the same word order as entered, but with the letters in each word reversed. For example, "this is a test" becomes "siht si a tset". *Hint:* modify program PGM8_2.ASM in section 8.3.
9. A problem in elementary algebra is to decide if an expression containing several kinds of brackets, such as, [,],{,},(,), is correctly bracketed. This is the case if (a) there are the same number of left and right brackets of each kind, and (b) when a right bracket appears, the most recent preceding unmatched left bracket should be of the same type. For example,

   $(a + [b - \{c \times (d - e)\}] + f)$ is correctly bracketed, but

   $(a + [b - \{c \times (d - e)]\} + f)$ is not

   Correct bracketing can be decided by using a stack. The expression is scanned left to right. When a left bracket is encountered, it is pushed onto the stack. When a right bracket is encountered,

the stack is popped (if the stack is empty, there are too many right brackets) and the brackets are compared. If they are of the same type, the scanning continues. If there is a mismatch, the expression is incorrectly bracketed. At the end of the expression, if the stack is empty the expression is correctly bracketed. If the stack is not empty, there are too many left brackets.

Write a program that lets the user type in an algebraic expression, ending with a carriage return, that contains round (parentheses), square, and curly brackets. As the expression is being typed in, the program evaluates each character. If at any point the expression is incorrectly bracketed (too many right brackets or a mismatch between left and right brackets), the program tells the user to start over. After the carriage return is typed, if the expression is correct, the program displays "expression is correct." If not, the program displays "too many left brackets". In both cases, the program asks the user if he or she wants to continue. If the user types 'Y', the program runs again.
Your program does not need to store the input string, only check it for correctness.

Sample execution:

```
ENTER AN ALGEBRAIC EXPRESSION:
(a + b)]TOO MANY RIGHT BRACKETS. BEGIN AGAIN!
ENTER AN ALGEBRAIC EXPRESSION
(a + [b - c] x d)
EXPRESSION IS CORRECT
TYPE Y IF YOU WANT TO CONTINUE:Y
ENTER AN ALGEBRAIC EXPRESSION:
[a + b x (c - d) - e}BRACKET MISMATCH. BEGIN AGAIN!
ENTER AN ALGEBRAIC EXPRESSION:
((a + [b - {c x (d - e) } ] + f)
TOO MANY LEFT BRACKETS. BEGIN AGAIN!
ENTER AN ALGEBRAIC EXPRESSION:
I'VE HAD ENOUGH
EXPRESSION IS CORRECT
TYPE Y IF YOU WANT TO CONTINUE:N
```

10. The following method can be used to generate random numbers in the range 1 to 32767.

    Start with any number in this range.
    Shift left once.
    Replace bit 0 by the XOR of bits 14 and 15.
    Clear bit 15.

    Write the following procedures:

    a. A procedure READ that lets the user enter a binary number and stores it in AX. You may use the code for binary input given in section 7.4.

    b. A procedure RANDOM that receives a number in AX and returns a random number in AX.

    c. A procedure WRITE that displays AX in binary. You may use the algorithm given in section 7.4.

    Write a program that displays a '?', calls READ to read a binary number, and calls RANDOM and WRITE to compute and display 100 random numbers. The numbers should be displayed four per line, with four blanks separating the numbers.